

On Autopilot? An Empirical Study of Human–AI Teaming and Review Practices in Open Source

Haoyu Gao
the University of Melbourne
Melbourne, Australia
haoyug1@student.unimelb.edu.au

Hao Guan
the University of Melbourne
Melbourne, Australia
guan.h@student.unimelb.edu.au

Peerachai Banyongrakkul
the University of Melbourne
Melbourne, Australia
pbanyongrakk@student.unimelb.edu.au

Mansoorreh Zahedi
the University of Melbourne
Melbourne, Australia
mansoorreh.zahedi@unimelb.edu.au

Christoph Treude
Singapore Management University
Singapore, Singapore
ctreude@smu.edu.sg

Abstract

Large Language Models (LLMs) increasingly automate software engineering tasks. While recent studies highlight the accelerated adoption of “AI as a teammate” in Open Source Software (OSS), developer interaction patterns remain under-explored. In this work, we investigated project-level guidelines and developers’ interactions with AI-assisted pull requests (PRs) by expanding the AIDev dataset to include finer-grained contributor code ownership and a comparative baseline of human-created PRs. We found that over 67.5% of AI-co-authored PRs originate from contributors without prior code ownership. Despite this, the majority of repositories lack guidelines for AI-coding agent usage. Notably, we observed a distinct interaction pattern: AI-co-authored PRs are merged significantly faster with minimal feedback. In contrast to human-created PRs where non-owner developers receive the most feedback, AI-co-authored PRs from non-owners receive the least, with approximately 80% merged without any explicit review. Finally, we discuss implications for developers and researchers.

CCS Concepts

• **Software and its engineering** → *Software organization and properties.*

Keywords

AI-Human Collaboration, Documentation, Code Review

ACM Reference Format:

Haoyu Gao, Peerachai Banyongrakkul, Hao Guan, Mansoorreh Zahedi, and Christoph Treude. 2026. On Autopilot? An Empirical Study of Human–AI Teaming and Review Practices in Open Source. In *23rd International Conference on Mining Software Repositories (MSR ’26)*, April 13–14, 2026, Rio

de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793573>

1 Introduction

LLM-powered coding agents are widely adopted across various software development tasks, including code review [16], code generation [7], and program repair [14]. This shift enables coding agents to take on significant roles as “coding teammates” within open source projects. Recently, Li et al. [15] curated a large-scale OSS pull request dataset capturing two distinct behaviours: developers delegating tasks entirely to AI agents (AI-only PRs) and developers co-authoring contributions alongside AI agents (Human+AI PRs).

Despite the increasing adoption of AI coding agents in OSS development, the ways in which developers prepare for and interact with these agents remain relatively understudied. Given the rapid expansion of AI-assisted coding, effective governance and monitoring have become increasingly critical, as the output of these agents directly impacts the integrity of the codebase.

In this paper, we bridge this gap by performing an empirical study of guideline documents and review practices at both the project and fine-grained developer levels. We expanded Li et al.’s dataset [15] with human-created PRs from the same period, alongside contributor ownership. Our analysis reveals that AI-assisted PRs have higher percentage of non-owner developers contribution, with over 67.5% of such PRs originate from contributors without prior code ownership. Yet, the majority (86.9%) of projects lack guidelines for AI agent usage. Notably, compared to Human-only PRs, Human+AI PRs receive significantly less feedback; contributors with less code ownership are approved faster and with lighter reviews than core members. These findings highlight the need for research into task allocation across human-AI paradigms. Our replication package is available at <https://zenodo.org/records/18005310>.

2 Related Work

Recent studies show that LLM-based coding tools are already widely used in open-source software (OSS) development [26]. Tufano et al.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR ’26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/26/04

<https://doi.org/10.1145/3793302.3793573>

[25] mined GitHub repositories to identify AI-assisted commits and pull requests, revealing that developers rely on AI tools for tasks including code generation, testing, and documentation. Champa et al. [3] further showed that developers treat LLMs as collaborative assistants rather than fully autonomous contributors. Similarly, Santos et al. [21] showed that developers use LLMs under human supervision, reporting productivity gains alongside concerns about correctness, limited context awareness, and ethical risks.

Other work has examined how AI-assisted contributions are reviewed and governed in OSS. Chouchen et al. [5] suggested that AI-involved pull requests tend to be larger and slower to review due to revision overhead. Ogenrwot et al. [19] found that developers frequently modify or partially reject AI-generated code due to quality and maintainability concerns. Meanwhile, Forsgren et al. [8] argued that developer productivity and collaboration should be assessed using multiple dimensions, including communication and review interactions, rather than speed alone. This perspective motivates our study, as little is known about differences in review and engagement patterns between AI-assisted and human-created contributions or about the presence of AI usage guidelines.

3 Study Design and Methodology

We structure our research into three research questions (RQs).

RQ1: To what extent has the reliance on AI-coding agents changed over time, and which types of contributors primarily adopt them?

RQ2: To what extent is the adoption of AI coding agents planned and discussed, or are there guidelines proposed for their usage?

RQ3: How do engagement and review practices for AI-assisted contributions differ from those for human-created contributions?

3.1 Data Collection

To establish a human baseline, we expanded the “popular” AIDev dataset by retrieving human-created PRs from the same repositories and timeframes. Specifically, we collected all non-AIDev PRs submitted between the first and last AI-related entry for each repository, yielding 243,306 PRs across 2,253 repositories. We then filtered out bots using Golzadeh et al.’s tool [10] and usernames ending in “bot” or “[bot]”, ensuring valid contributors were preserved. The final dataset comprises 174,567 “Human-only” PRs across 2,134 repositories, and we make it publicly available.

To mine AI agent usage guidelines, we collected project artefacts reflecting planning or governance decisions. We focused on common files (README, CONTRIBUTING) and, following a manual inspection of 100 repositories, included Code of Conduct and Pull Request templates. We then performed a temporal collection anchored at each repository’s first AI-co-authored PR, capturing an initial version (one month prior) and all updates until the last agentic PR. Table 1 (left) summarises these artefacts, distinguishing pre-adoption guidance from post-adoption updates.

To determine contributors’ past ownership in the projects, we adopted the concept of code ownership. Following Lulla et al. [17], we used “Git Blame” to establish code ownership, as it provides an owner for each line in a file. We calculated the contribution by first checking out the commit immediately preceding the first AI-co-authored PR. Subsequently, we calculated the contribution factors

Table 1: Results for RQ2.

	Data Collection			Keyword Filtering		
	# Repos	# Files	# Patches	# Repos	# Files	# Patches
Ctb	1,183	1,183	350	73	69	15
Rmd	2,706	2,706	4,621	512	478	797
CoC	800	800	22	4	3	1
PRT	727	727	74	8	8	0

Ctb = CONTRIBUTING, Rmd = README, CoC = Code of Conduct, PRT = PR Template.

of a contributor across all traceable files as follows: $C_i = \sum_{j \in \mathcal{F}} \frac{l_{ij}}{l_j}$ where C_i stands for the contribution of each contributor, and $\mathcal{F}$ is the set for all files that “Git Blame” can trace the contributors.

3.2 Data Analysis

To answer RQ1 about the reliance of AI coding agent, we first analysed longitudinal trends by comparing the monthly volume of AI-co-authored and Human-only PRs across active repositories. Subsequently, for each repository, we calculated contributor ownership, ranked developers, and analysed both contributor distribution and PR volume across ranking tiers.

For RQ2 on guidelines, we screen the collected artefacts for AI agent usage guidance using keyword filtering. Specifically, we searched for the five AI agent names, plus “agent” and “agentic”. We analysed full content for initial versions and patches for updates, lowercasing, tokenising, and lemmatising text before matching. Table 1 reports the results. We sampled up to 100 items per artefact for manual annotation, achieving strong agreement (Cohen’s $\kappa = 0.85$) among two raters [18]. We resolved conflicts via discussion. Lastly, we grouped them into themes to investigate their nature.

To answer RQ3 about review practices, we stratified the AIDev dataset into “AI-only PRs” and “Human+AI PRs”. Leveraging the results from our bot-detection process, we identified authors classified as “bot”, and labelled their contribution as “AI-only PRs”. The remaining entries in AIDev are labelled as “Human+AI PRs”. We compared these against the “Human-only PRs” from our expanded dataset, excluding any PRs that were not yet closed. These datasets are directly comparable as they cover the same repositories over an identical time period. Furthermore, for “Human+AI PRs” and “Human-only PRs”, we partitioned them based on the creators’ ownership. We divided contributors into three cohorts: those ranked in the top 50% of code ownership, lower 50% (among contributors with ownership), and those with no prior code ownership, based on the calculation we mentioned in Section 3.1. These groups are abbreviated as “Top50%”, “Low50%”, and “Out”, respectively.

Following this preprocessing, we calculated the average number of comments, code reviews, automated feedback, and time to close for each cohort. We also calculated the total number of PRs that received no feedback other than from AI bots or the PR creators themselves. These metrics serve as a proxy for the review effort spent on different cohorts across three development paradigms.

4 Results and Discussion

This section contains the result for three RQs and the discussion.

RQ1: AI Coding Agents Adoption. First, we present trends in AI-assisted PR adoption. Figure 1 displays the average monthly

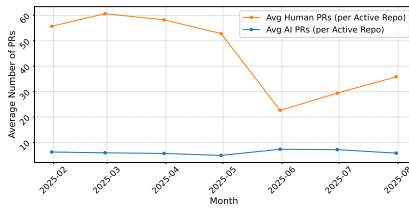


Figure 1: Average Human vs AI PRs Across the Active Months

volume of AI-assisted versus Human-only PRs. As shown, the frequency of AI-assisted PRs remains low, despite a minor increase since June 2025. However, as more repositories adopt AI agents, the volume of Human-only PRs has declined since June 2025. This divergence has resulted in a higher ratio of AI co-authorship compared to the period preceding this turning point. Next, we analysed contributor cohorts for both types. A comparison of unique counts reveals the disparity: while human-created PRs involve 24,000 contributors, AI-assisted PRs originating from the same repositories only account for 1,797 contributors within the same repositories.

Regarding contributor ownership, 32.5% of AI-assisted PRs (excluding AI-only PRs) involve authors with prior code ownership, compared to only 39.0% of Human-only PRs. This indicates slightly higher percentage of the AI-assisted PR creators are newcomers to the project, and the vast majority originate from contributors without ownership. Figure 2 illustrates the relationship between developers' code ownership and their contribution volume. We defined ownership by rank, where a rank of 1 denotes the contributor with the highest code ownership in the cohort. The data reveals distinct paradigms between human and AI workflows. In the human context, developers with ownership contribute fewer PRs, while the distribution remains diverse, more evenly across the contributors. In contrast, AI-assisted is more more utilised by the top contributors. However, usage is not exclusive to developers with ownership; the majority of users (67.5%) initiating AI-assisted PRs possess no prior code ownership, suggesting adoption is concentrated at the two ownership extremes.

RQ1 Summary: The recent surge in AI adoption accompanies a decrease in Human-only PRs. Meanwhile, larger proportion of AI-assisted PRs are created by developers without prior code ownership compared Human-only PRs (67.5% vs 61.0%). Additionally, top-ranking contributors tend to create more AI-assisted PRs.

RQ2: AI Coding Agent Guidelines. Following keyword filtering and sampling, we analysed 196 artefacts (116 initial, 80 patches) from 145 repositories. Manual annotation reveals only 21 (10.7%) contain AI agent guidelines, appearing most commonly in CONTRIBUTING files (13/21, 62.5%) and PR templates (5/21, 32.8%). At the repository level, 13.1% (19/145) possess at least one artefact mentioning AI usage. Among 19 repositories that have the guidelines, 15 (78.9%) mentioned AI agent usage before their first AI-assisted PR, suggesting planned adoption. Among these, only one repository (6.7%) updates any related documentation after adoption; however, none introduce new or revised the guidance in later patches. In contrast, four (21.1%) introduce guidance only after AI-assisted PRs appear, reflecting reactive governance.

To further characterise these 21 guideline items, we grouped them into themes based on the intent and focus of the guidance. We observed that the most common guidance focuses on practical rules for AI contributions. These include disclosure requirements, such as “prefix your commit message with AI:” [20] or “Please disclose any use of LLMs” [12]. Another frequent group provides how-to guidance for AI tools, pointing contributors to rules, prompts, or configurations, for instance “find additional prompts in .github/prompts or type #prompt in Copilot” in [22]. A smaller but notable group sets boundaries on AI involvement, emphasising human responsibility, for example “an AI summary isn’t enough by itself” stated in [11] or that certain files should be “written by a human (with minimal help from a language model)” [13]. Finally, a few items address community norms, embedding AI use into codes of conduct, for example by giving “guidelines to follow when using generative AI to answer questions in the community” [6].

RQ2 Summary: Guidance on AI agent usage is rare, indicating that most projects adopt AI agents in an unplanned manner. When present, guidance is typically introduced before the first AI-coauthored PR while a few add guidance only after AI usage begins. Most guidance focuses on disclosure of AI use.

RQ3: AI-Human Interaction Patterns. We divided the data into three paradigms: “AI-only PR”, “Human+AI PR”, and “Human-only PRs”. We then divided the latter two based on the contributors' ownership to the project. We further stratified the latter two by contributor ownership. Using prior results, we classified contributors as top-50% or lower-50% (among those with ownership), versus those with no ownership. Then we analysed the PR activities for these paradigms. Table 2 shows the results.

The table shows Human-only PRs achieved the highest merge rates. Meanwhile, AI-only PRs received only 55.3%, indicating nearly half were rejected. A closer look at the merge rates for contributors with different levels of ownership reveals opposite patterns for Human+AI and Human-only PRs. For Human-only PRs, the top 50% contributors received the highest merge rate of 89.7%, dropping to 78.4% for non-owners. In contrast, for Human+AI PRs, developers with code ownership received merge rates of 73.2% and 73.3% for the top and lower 50% respectively, while those without code ownership received a highest 84.8% merge rate.

For AI-only PRs, they took on average roughly the same amount of time to be merged, but much less time to be closed without merging. Only 1.3% are closed without any comment or review from any contributors in the project, and on average the merged PRs received more comments and reviews by the contributors. For Human+AI PRs, we see significantly less developer effort; the majority (79.0% merged, 59.9% closed) received no external reviews or comments. Consequently, the average amount of comments and reviews received are below one per PR for all contributor groups. Still, feedback decreases as contributor ownership lowers. Regarding the average time to close, Human+AI PRs are faster than the other two paradigms, with the merging time goes even below one day for the lower and non-owner developers.

Human-only PRs serve as a baseline of the traditional development patterns in OSS projects. Overall, 19.8% and 25.5% of the merged and unmerged PRs are closed without any human comment

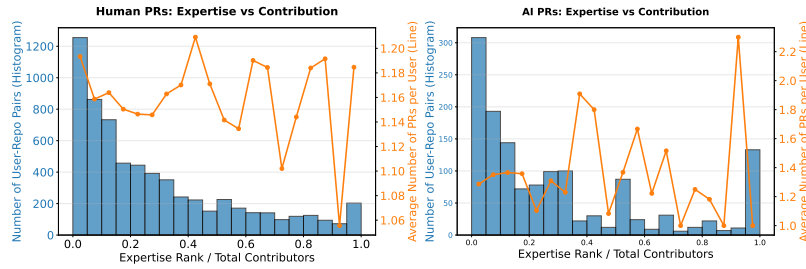


Figure 2: Contributors' Ownership Distribution and Contribution Volume for Human and AI PRs

Table 2: Review Statistics for Different Development Paradigms across Contributors with Different Ownership

PR Paradigm		# Entries	Comments	Reviews	Auto. Feedback	# No Human	Time to Close
AI-only PR	Merged	4,751	3.25	3.21	1.65	83	2.42 days
	Unmerged	3,845	2.65	0.89	1.45	31	5.53 days
Human+AI-Top50%	Merged	4,953	0.68	0.62	0.18	3,004	1.03 days
	Unmerged	1,815	0.99	0.40	0.23	842	5.39 days
Human+AI-Low50%	Merged	692	0.62	0.03	0.22	398	0.62 days
	Unmerged	252	0.72	0.34	0.08	129	2.26 days
Human+AI-Out	Merged	13,618	0.18	0.15	0.05	11,817	0.18 days
	Unmerged	2,435	0.52	0.14	0.18	1,727	2.31 days
Human-only-Top50%	Merged	54,261	2.58	4.98	0.53	7,971	2.32 days
	Unmerged	6,205	3.05	4.21	0.53	1,429	15.97 days
Human-only-Low50%	Merged	5,385	2.53	4.76	0.29	1,239	1.82 days
	Unmerged	873	3.25	4.76	0.40	188	17.63 days
Human-only-Out	Merged	80,396	2.99	5.52	0.70	18,497	3.68 days
	Unmerged	22,130	3.77	5.11	0.69	5,823	18.86 days

or review other than the contributor who created the PR. Meanwhile, much higher engagement is witnessed, with merged PRs averaging 4.76 to 5.52 reviews. Contributors who do not have past authorship to the codebase also receive higher amount of reviews and comments. The trend is also in the average time to close a PR. With the average time much higher than Human+AI PRs, and the PRs from contributors who do not have ownership to the codebase also take longer time to be merged.

RQ3 Summary: We observe a converse patterns in Human+AI development versus Human-only PRs: contributions from low-ownership developers are approved faster and with less scrutiny than even those from core members.

Discussion and Implications. While 67.5% of Human+AI PRs originate from non-owners, guidelines for AI agents remain scarce, echoing prior guidance gaps [1]. Therefore, similar to other documentation maintenance [9], project maintainers should make efforts in regulating the usage of AI coding agent and constantly adapt the instruction for newly-emerged issues.

Our analysis of Human+AI PRs yields counter-intuitive results: most Human+AI PRs are closed without explicit review. Even when reviewed, non-owner contributors tend to receive less feedback compared to core members, which is completely opposite to the pattern observed in Human-only PRs. Confounding factors of this phenomenon could relate to the variability of the development tasks for different development paradigms. Due to space constraints, we did not analyse the reasons behind it. We hypothesise that this may stem from Human+AI PRs primarily targeting low-hanging fruit within the projects. Future research should investigate the types of tasks allocated to contributors with varying ownership across

Human+AI, AI-only, and Human-only paradigms. Additionally, studies should examine whether code quality might be compromised due to less rigorous guardrails for Human+AI PRs. Finally, it is worth exploring whether automatic task allocation can be achieved across different development paradigms similar to prior reviewer assignments [23]; specifically for newcomers, researchers could investigate whether the AI-aided assignment of entry-level tasks facilitates better project onboarding [24].

5 Threats to Validity

Firstly, we assessed developers' profile solely based on code ownership. Incorporating additional signals, such as developer profile metadata or comprehensive commit histories, could provide deeper insights. Second, multiple usernames can link to the same users, and we did not resolve the alias issue [2], which could impact the result. Third, we leveraged the bot-detection algorithm from Golzadeh et al. [10], which relies on comment patterns. Consequently, this approach may fail to identify bots in repositories with limited activity, affecting the precise classification of PRs. Finally, our analysis focused on standard contribution files which might not be comprehensive; future studies could expand this to specialised documentation such as Agent.md [4].

6 Ethical Implications

The code ownership data comprises non-anonymised developer usernames.

7 Conclusion

We investigated developer interactions with AI-assisted PRs and guidelines. We mined documentation and expanded the AIDev dataset with contributor ownership and a human baseline. We found 67.5% of AI-assisted originate from contributors without code ownership. Yet, 86.9% of repositories lack guidelines for AI agent usage. Furthermore, Human+AI PRs merge significantly faster with minimal feedback. Notably, non-owner PRs receive the least feedback, with approximately 80% of merged without explicit reviews, contrasting human PRs where non-owner developers receive the most. We conclude with implications for developers and researchers.

References

- [1] Peerachai Banyongrakkul, Mansoor Zahedi, Patanamon Thongtanunam, Christoph Treude, and Haoyu Gao. 2025. From Release to Adoption: Challenges in Reusing Pre-Trained AI Models for Downstream Developers. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–13. doi:10.1109/ICSME64153.2025.00022
- [2] Stefano Campanella and Michele Lanza. 2024. Hidden in the code: Visualizing true developer identities. In *2024 IEEE Working Conference on Software Visualization (VISST)*. IEEE, 24–35.
- [3] Arifa Islam Champa, Md Fazle Rabbi, Costain Nachuma, and Minhaz F. Zibran. 2024. ChatGPT in Action: Analyzing Its Use in Software Development. In *Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 182–186. doi:10.1145/3643991.3645077
- [4] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjansith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E Hassan, et al. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. *arXiv preprint arXiv:2511.12884* (2025).
- [5] Moataz Chouchen, Narjes Bessghaier, Mahi Begoug, Ali Ouni, Eman Alomar, and Mohamed Wiem Mkaouer. 2024. How Do Software Developers Use ChatGPT? An Exploratory Study on GitHub Pull Requests. In *Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 212–216. doi:10.1145/3643991.3645084
- [6] community/community. 2025. *CODE_OF_CONDUCT.md*. https://github.com/community/community/blob/98443f20fec26b5619de086f803b91f5903629c/CODE_OF_CONDUCT.md (Accessed: 2025-12-12).
- [7] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* (2024).
- [8] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Tom Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There's more to it than you think. *ACM Queue* 19, 1 (feb 2021), 20–48. <https://www.microsoft.com/en-us/research/publication/the-space-of-developer-productivity-theres-more-to-it-than-you-think/>
- [9] Haoyu Gao, Christoph Treude, and Mansoor Zahedi. 2025. Adapting Installation Instructions in Rapidly Evolving Software Ecosystems. *IEEE Transactions on Software Engineering* (2025).
- [10] Mehdi Golzadeh, Alexandre Decan, Damien Legay, and Tom Mens. 2021. A ground-truth dataset and classification model for detecting bots in GitHub issue and PR comments. *Journal of Systems and Software* 175 (may 2021). doi:10.1016/j.jss.2021.110911
- [11] HumanSignal/label-studio. 2025. *.github/pull_request_template.md*. https://github.com/HumanSignal/label-studio/blob/1543b6ccaff8880c5ae2ff6ffac3db1e00f0761/.github/pull_request_template.md (Accessed: 2025-12-12).
- [12] igraph/igraph. 2025. *.github/pull_request_template.md*. https://raw.githubusercontent.com/igraph/igraph/8baaad8c6b9640a8c2ae875ad2066e027d23fa41/.github/pull_request_template.md (Accessed: 2025-12-12).
- [13] Hlaude institute/terminal bench. 2025. *.github/pull_request_template.md*. https://github.com/laude-institute/terminal-bench/blob/ada99ca331dc93c08fde181484361df1e1a12f68/.github/pull_request_template.md (Accessed: 2025-12-12).
- [14] Thanh Le-Cong, Bach Le, and Toby Murray. 2024. Semantic-guided Search for Efficient Program Repair with Large Language Models. *arXiv preprint arXiv:2410.16655* (2024).
- [15] Hao Li, Haoxiang Zhang, and Ahmed E Hassan. 2025. The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003* (2025).
- [16] Hong-Yi Lin, Chunhua Liu, Haoyu Gao, Patanamon Thongtanunam, and Christoph Treude. 2025. Codereviewqa: The code review comprehension assessment for large language models. In *Findings of the Association for Computational Linguistics: ACL* 2025. 9138–9166.
- [17] Jai Lal Lulla, Raula Gaikovina Kula, and Christoph Treude. 2025. Automated Code Review Assignments: An Alternative Perspective of Code Ownership on GitHub. *arXiv preprint arXiv:2512.05551* (2025).
- [18] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.
- [19] Daniel Ogenrwot and John Businge. 2024. PatchTrack: Analyzing ChatGPT's Impact on Software Patch Decision-Making in Pull Requests. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 2480–2481. doi:10.1145/3691620.3695338
- [20] rsyslog/rsyslog. 2025. *CONTRIBUTING.md*. <https://github.com/rsyslog/rsyslog/blob/26bb550822cc9ecbd5bb309a4362cfa9496facc/CONTRIBUTING.md> (Accessed: 2025-12-12).
- [21] Italo Santos, Cleyton Magalhaes, and Ronnie de Souza Santos. 2025. Model-Assisted and Human-Guided: Perceptions and Practices of Software Professionals Using LLMs for Coding. *arXiv:2510.09058 [cs.SE]* <https://arxiv.org/abs/2510.09058>
- [22] Azure/azure sdk-for rust/. 2025. *CONTRIBUTING.md*. <https://github.com/Azure/azure-sdk-for-rust/blob/12285bce0eed33643bad6ab6c28ff41435c0b64a/CONTRIBUTING.md> (Accessed: 2025-12-12).
- [23] Daricélio M Soares, Manoel L de Lima Júnior, Alexandre Plastino, and Leonardo Murta. 2018. What factors influence the reviewer assignment to pull requests? *Information and Software Technology* 98 (2018), 32–43.
- [24] Igor Steinmacher, Tayana Uchoa Conte, Christoph Treude, and Marco Aurélio Gerosa. 2016. Overcoming open source project entry barriers with a portal for newcomers. In *Proceedings of the 38th International Conference on Software Engineering*. 273–284.
- [25] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. 2024. Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study. In *Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24)*. Association for Computing Machinery, New York, NY, USA, 571–583. doi:10.1145/3643991.3644918
- [26] Kaixin Wang, Tianlin Li, Xiaoyu Zhang, Chong Wang, Weisong Sun, Yang Liu, and Bin Shi. 2025. Software Development Life Cycle Perspective: A Survey of Benchmarks for CodeLLMs and Agents. doi:10.48550/arXiv.2505.05283